

Prof. Stefan Göller
Anna Maiworm

Einführung in die Informatik

WiSe 2025/2026

Hausaufgabenblatt 08, Abgabe am 12.01.26 um 12:00 Uhr

Allgemeine Hinweise

- Wenn die Hausaufgabe exakte Benennungen von Dateien, Funktionen, Variablen, etc. vorgibt, dann müssen Sie sich an diese Vorgabe halten. Falls nicht wird Ihre Abgabe nicht oder nur mit Punktabzügen korrigiert.
- Gleiches gilt, wenn die Aufgabe konkrete Anweisungen für `input()` oder `print()` vorgibt.
- Ihr Programm darf keine zusätzlichen `print()` Befehle enthalten, welche in der Aufgabe nicht gefordert sind. Ist eine Ausgabe mit `print()` gefordert, so geben Sie, sofern nicht anders vorgegeben, nur das geforderte Ergebnis und **keinen** zusätzlichen Text aus.
- Benutzen Sie **keine Python-Imports**, außer dies ist in der Aufgabenstellung explizit erlaubt.
- Beachten Sie die Anweisungen, die ggf. zusätzlich in den jeweiligen Aufgaben gegeben sind.
- Geben Sie für jede Aufgabe eine separate Datei ab. Benennen Sie diese Datei als `bXXaY.py`, wobei XX die Blattnummer und Y die Aufgabennummer ist.
- Schreiben Sie die Namen aller Gruppenmitglieder als Kommentar in die erste Zeile jeder Datei.
- Programmcode muss möglichst einfach und gut lesbar sein, um die Korrektur zu erleichtern. Code, der nicht oder nur schwer lesbar ist, kann zu Punktabzug führen.

Hausaufgabe 1 (2 Punkte):

Schreiben Sie eine Funktion **umkehren**, welche als Eingabe ein Wörterbuch **wb** erwartet, in dem die Schlüssel Integer-Werte sind. Zurückgegeben werden soll ein neues Wörterbuch, das die Schlüssel und Werte von **wb** vertauscht. Falls es in **wb** zwei verschiedene Schlüssel gibt, die auf den gleichen Wert verweisen, so soll der kleinste Schlüssel als neuer Wert benutzt werden.

Beispiel: `umkehren({2:3, 4:"ab", 5:True, 9:"ab"})` liefert als Rückgabe ein Wörterbuch der Form `{3:2, "ab":4, True:5}`.

Ihre Funktion **umkehren** darf **keine Nebeneffekte** auf das eingegebene Wörterbuch haben, d.h. durch die Ausführung Ihrer Funktion, darf das eingegebene Wörterbuch nicht verändert werden.

Hausaufgabe 2 (3 Punkte):

Schreiben Sie eine Funktion **zusammenfuehren**, welche als Eingabe zwei Wörterbücher erwartet. Zurückgegeben werden soll ein neues Wörterbuch, das die beiden eingegebenen Wörterbücher zusammenführt. Wenn ein Schlüssel in beiden eingegebenen Wörterbüchern vorkommt, so soll dieser Schlüssel im zurückgegebenen Wörterbuch auf das Tupel der beiden ursprünglichen Werte verweisen.

Beispiel:

`zusammenfuehren({"apfel":12, "birne":7}, {"apfel":"Kl.B", "kirsche":"Kl.A"})`

liefert als Rückgabe ein Wörterbuch der Form

`{"apfel":(12,"Kl.B"), "birne":7, "kirsche":"Kl.A"}`

Ihre Funktion **zusammenfuehren** darf **keine Nebeneffekte** auf die eingegebenen Wörterbücher haben, d.h. durch die Ausführung Ihrer Funktion, dürfen die eingegebenen Wörterbücher nicht verändert werden.

Hausaufgabe 3 (2 Punkte):

Schreiben Sie eine Funktion `wort_paare`, welche als Eingabe einen String erwartet, der aus mehreren Wörtern besteht, die jeweils durch genau ein Leerzeichen getrennt sind. Zurückgegeben werden soll ein Wörterbuch, welches angibt wie oft Wörter aufeinander folgen, d.h. als Schlüssel sollen alle Tupel (w_1, w_2) vorkommen, sodass die Wörter w_1 und w_2 in der Eingabe hintereinander vorkommen. Der zugeordnete Wert soll angeben, wie oft diese Wortkombination in der Eingabe vorkommt.

Beispiel:

```
wort_paare("der Hund läuft und der Hund läuft schnell und der Hund bellt")
```

liefert als Rückgabe ein Wörterbuch der Form

```
{("der", "Hund"):3, ("Hund", "läuft"):2, ("läuft", "und"):1, ("und", "der"):2,
 ("läuft", "schnell"):1, ("schnell", "und"):1, ("Hund", "bellt"):1}
```

Hausaufgabe 4 (4 Punkte):

Schreiben Sie eine **rekursive** Funktion `gitter_wege`, welche als Eingabe zwei positive Integer-Werte m und n , sowie ein Wörterbuch `wb`, erwartet. Von der Funktion `gitter_wege` berechnet und zurückgegeben werden soll die Anzahl der möglichen Wege die es gibt, in einem Gitter der Höhe m und der Breite n , von oben links nach unten rechts zu laufen. Dabei darf man in jedem Schritt nur nach unten oder rechts laufen.

Fordern Sie im Hauptprogramm die zwei Werte m und n vom Nutzer an, rufen Sie `gitter_wege` mit den Eingaben m , n , `{}` auf (m und n als Integer) und geben die Rückgabe auf der Konsole aus.

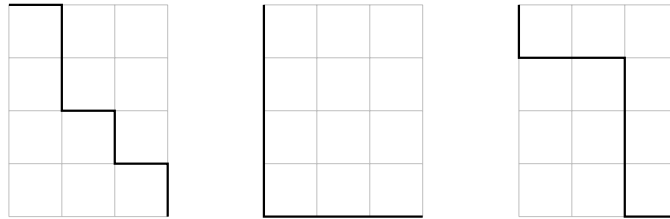
Das übergebene Wörterbuch `wb` soll genutzt werden, um Ihr Programm schneller zu machen. D.h. setzen Sie das Wörterbuch ein, sodass bereits berechnete rekursive Aufrufe nicht erneut berechnet werden. Die Schlüssel dieses Wörterbuchs sollten Tupel der Form (m', n') sein, wobei m' , n' positive Ganzzahlen sind.

Beispiele: Im Folgenden sind alle zulässigen Wege im 3,3-Gitter dargestellt:



Die Dimensionen beziehen sich also auf die Anzahl der Gitterpunkte, nicht auf die Anzahl der Kästchen.

Die folgenden Wege sind z.B. zulässig auf dem 5,4-Gitter:



Die Rückgabe von `gitter_wege(5,4,{})` lautet 35.

Hausaufgabe 5 (6 + 3 Punkte):

In dieser Aufgabe sollen Sie die zentrale Funktion `muenz_wechsel` für ein Programm schreiben, das dem Nutzer anzeigt wie er einen gewünschten Geldbetrag in Euro- und Cent-Münzen wechseln kann. Hierzu soll die Funktion Rekursion verwenden.

Nutzen Sie dazu in Ihrer Datei das folgende Hauptprogramm:

```
betrag = int(input("Geben Sie einen Betrag in Cent ein: "))
min_muenze = int(input("Geben Sie die kleinste Münze ein (in Cent): "))
out = muenz_wechsel(betrag, min_muenze)
for liste in out:
    print(liste)
```

Der Nutzer wird also zu zwei Eingaben aufgefordert: Zuerst der Betrag der gewechselt werden soll, angegeben in Cent (also z.B. 511 für 5,11 Euro). Anschließend den Wert der kleinsten Münze, die verwendet werden soll, ebenfalls in Cent (also z.B. 20 für die 20-Cent-Münze)

Die Münzen die im Allgemeinen verwendet werden können sind die Euro-Münzen, also 1, 2, 5, 10, 20, 50 Cent, 1 Euro, 2 Euro.

Ihre Funktion `muenz_wechsel` soll als Rückgabe eine Liste von Listen liefern, die alle Möglichkeiten angibt, den gewünschten Betrag, mit den gewünschten Münzen, zu wechseln. Diese zurückgegebene Liste von Listen wird durch das Hauptprogramm zeilenweise auf der Konsole ausgegeben. Gibt der Nutzer z.B. 120 und 20 ein, so soll man die folgende Konsolenausgabe erhalten:

```
['20 Cent', '20 Cent', '20 Cent', '20 Cent', '20 Cent', '20 Cent']  
['50 Cent', '50 Cent', '20 Cent']  
['1 Euro', '20 Cent']
```

Wie Sie sehen, soll jede Münzen-Kombination nur genau einmal auftauchen (z.B. soll `['20 Cent', '1 Euro']` nicht auch noch ausgegeben werden). Dies können Sie erreichen, indem Sie die Listen absteigend sortiert erstellen, d.h. in den rekursiven Aufrufen werden jeweils nur Münzen verwendet, die einen Wert kleiner oder gleich der Münze haben, die gerade verwendet wurde. Nutzen Sie hierfür eine Hilfsfunktion, die die Rekursion übernimmt, als Eingabe zusätzlich einen Integer-Wert `max_muenze` erhält und die Münzwechsel-Möglichkeiten berechnet, die man hat, wenn man nur Münzen mit Wert zwischen `min_muenze` und `max_muenze` verwendet.

Wie im Beispiel zu sehen, sollen in der Rückgabe von `muenz_wechsel`, die Münzen als String mit Einheit angegeben sein (also z.B. `'5 Cent'` oder `'1 Euro'`).

Wenn der Nutzer seine Eingaben so wählt, dass es nicht möglich ist den eingegebenen Betrag, mit den gewünschten Münzen, zu wechseln (z.B. bei 21 und 5), darf sich Ihre Funktion beliebig verhalten.

- (a) Schreiben Sie die Funktion `muenz_wechsel` mit Ihrer beschriebenen Hilfsfunktion.
- (b) Fügen Sie Ihrer Hilfsfunktion ein Wörterbuch als Eingabe hinzu und nutzen Sie dieses, um Ihr Programm schneller zu machen. D.h. setzen Sie das Wörterbuch ein, sodass bereits berechnete rekursive Aufrufe nicht erneut berechnet werden. Die Schlüssel dieses Wörterbuchs sollten Tupel der Form `(betrag, min_muenze, max_muenze)` sein.

Geben Sie Ihre Lösungen für (a) und (b) in separaten Dateien mit den Namen `b08a5a.py` und `b08a5b.py` ab. In beiden Dateien soll das Hauptprogramm, exakt wie hier angegeben, enthalten und Ihre Funktion `muenz_wechsel` benannt sein.

Präsenzaufgaben

Präsenzaufgabe 1:

Im Folgenden soll ein Buchungssystem für ein Kino implementiert werden. Definieren Sie dazu im Hauptprogramm ein Wörterbuch `kino`, dessen Werte wiederum Wörterbücher sind und welches die folgende Form hat:

```
{"Saal1":{"18:00":50, "20:00":30}, "Saal2":{"18:30":40, "20:00":35, "21:30":45}}
```

Hierbei geben die Integer-Werte jeweils die Anzahl belegter Plätze für den entsprechenden Saal, zur entsprechenden Zeit an. Die Anzahl der Säle, die Uhrzeiten und die Belegungszahlen können dabei vom Beispiel abweichen, die restliche Struktur ist aber wie hier angegeben.

Definieren Sie im Hauptprogramm außerdem ein Wörterbuch `plaetze_gesamt`, welches für jeden Saal angibt, wie viele Plätze der Saal insgesamt hat, z.B. `{"Saal1":60, "Saal2":50}`.

- (a) Schreiben Sie eine Funktion `vorstellung_hinzufuegen`, welche als Eingabe einen Saal und eine Uhrzeit (beide als String, wie in `kino`) erwartet. Die Funktion soll den Effekt haben, dass dem Wörterbuch `kino` aus dem Hauptprogramm eine entsprechende Vorstellung hinzugefügt wird, für die noch keine Plätze gebucht sind. Zurückgegeben werden soll `None`.

Falls bereits eine Vorstellung mit den selben Daten existiert, so soll stattdessen auf der Konsole "**Vorstellung schon vorhanden.**" ausgegeben werden.

Falls der eingegebene Saal noch nicht in `kino` existiert, ist dies jedoch kein Hindernis, es soll auch dann ein entsprechender Eintrag angelegt werden.

- (b) Schreiben Sie eine Funktion `plaetze_buchen`, welche als Eingabe einen Saal, eine Uhrzeit (beide als String, wie in `kino`) und eine Anzahl von Plätzen, die gebucht werden sollen (als Integer), erwartet. Die Funktion soll den Effekt haben, dass das Wörterbuch `kino` aus dem Hauptprogramm entsprechend aktualisiert wird. Zurückgegeben werden soll `None`.

Beispiel: `plaetze_buchen("Saal2", "18:30", 3)`, mit Hauptprogramm wie in den Beispielen oben, erhöht die entsprechende Belegungszahl im Wörterbuch `kino` von 40 auf 43.

Falls zu den eingegebenen Daten im Wörterbuch `kino` kein Eintrag existiert, so soll stattdessen auf der Konsole "**Keine passende Vorstellung vorhanden.**" ausgegeben werden.

Falls in der Vorstellung zu den eingegebenen Daten nicht mehr genug Plätze frei sind, so soll auf der Konsole "**Nicht genügend Plätze frei.**" ausgegeben werden. Sie dürfen davon ausgehen, dass der eingegebene Saal im Wörterbuch `plaetze_gesamt` als Schlüssel vorkommt.

Präsenzaufgabe 2:

Wir sagen, dass ein String s' ein *Unterstring* vom String s ist, wenn man s' erhalten kann, indem man irgendwelche Zeichen aus s weglässt (oder keins).

Beispiel: Alle Unterstrings von "abc" sind "abc", "bc", "ac", "ab", "c", "b", "a", "".

Schreiben Sie eine Funktion `palindrom_unterstrings`, welche als Eingabe einen String s , sowie ein Wörterbuch wb , erwartet. Berechnet und zurückgegeben werden soll eine Liste aller Unterstrings von s , die Palindrome sind.

Dazu sollen die Unterstrings auf rekursive Weise durchlaufen werden und für jeden Unterstring soll getestet werden, ob es sich um ein Palindrom handelt. Dieser Test muss nicht unbedingt rekursiv implementiert werden.

Fordern Sie im Hauptprogramm den String s vom Nutzer an, rufen Sie `palindrom_unterstrings` mit den Eingaben s , $\{\}$ auf und geben die Rückgabe auf der Konsole aus.

Das übergebene Wörterbuch wb soll genutzt werden, um Ihr Programm schneller zu machen. D.h. setzen Sie das Wörterbuch ein, sodass bereits berechnete rekursive Aufrufe nicht erneut berechnet werden. Die Schlüssel dieses Wörterbuchs sollten Strings sein.

In der Rückgabe-Liste soll kein Eintrag doppelt vorkommen.

Beispiel: `palindrom_unterstrings(abcba, {})` liefert als Rückgabe eine Liste mit den folgenden Einträgen (Reihenfolge irrelevant):

```
['abcba', 'a', '', 'b', 'c', 'bb', 'bcb', 'aba', 'aa', 'aca', 'abba']
```

Präsenzaufgabe 3:

Erinnern Sie sich an die Collatz-Folge aus Aufgabenblatt 6 Präsenzaufgabe 6: Für jede positive Ganzzahl n ist die nächste Zahl in der Collatz-Folge:

- $n/2$, falls n gerade ist
- $3 \cdot n + 1$, falls n ungerade ist

- (a) Schreiben Sie eine **rekursive** Funktion `collatz_zaeher`, welche als Eingabe eine positive Ganzzahl n erwartet. Berechnet und zurückgegeben werden, soll die Länge der Collatz-Folge, gestartet bei n , bis einschließlich zum ersten Auftreten einer 1.
- (b) Schreiben Sie eine Funktion `laengste_folge`, welche die Funktion `collatz_zaeher` verwendet und unter den Ganzzahlen $1 - 1.000.000$ diejenigen findet, deren Collatz-Folge am längsten ist. Diese sollen auf der Konsole ausgegeben werden. Die Rückgabe der Funktion ist `None`.

Fügen Sie den Funktionen außerdem, auf geeignete Weise, die Verwendung eines Wörterbuches hinzu, um Ihr Programm schneller zu machen. D.h. setzen Sie das Wörterbuch ein, sodass bereits berechnete rekursive Aufrufe nicht erneut berechnet werden. Die Schlüssel dieses Wörterbuchs sollten positive Ganzzahlen sein.